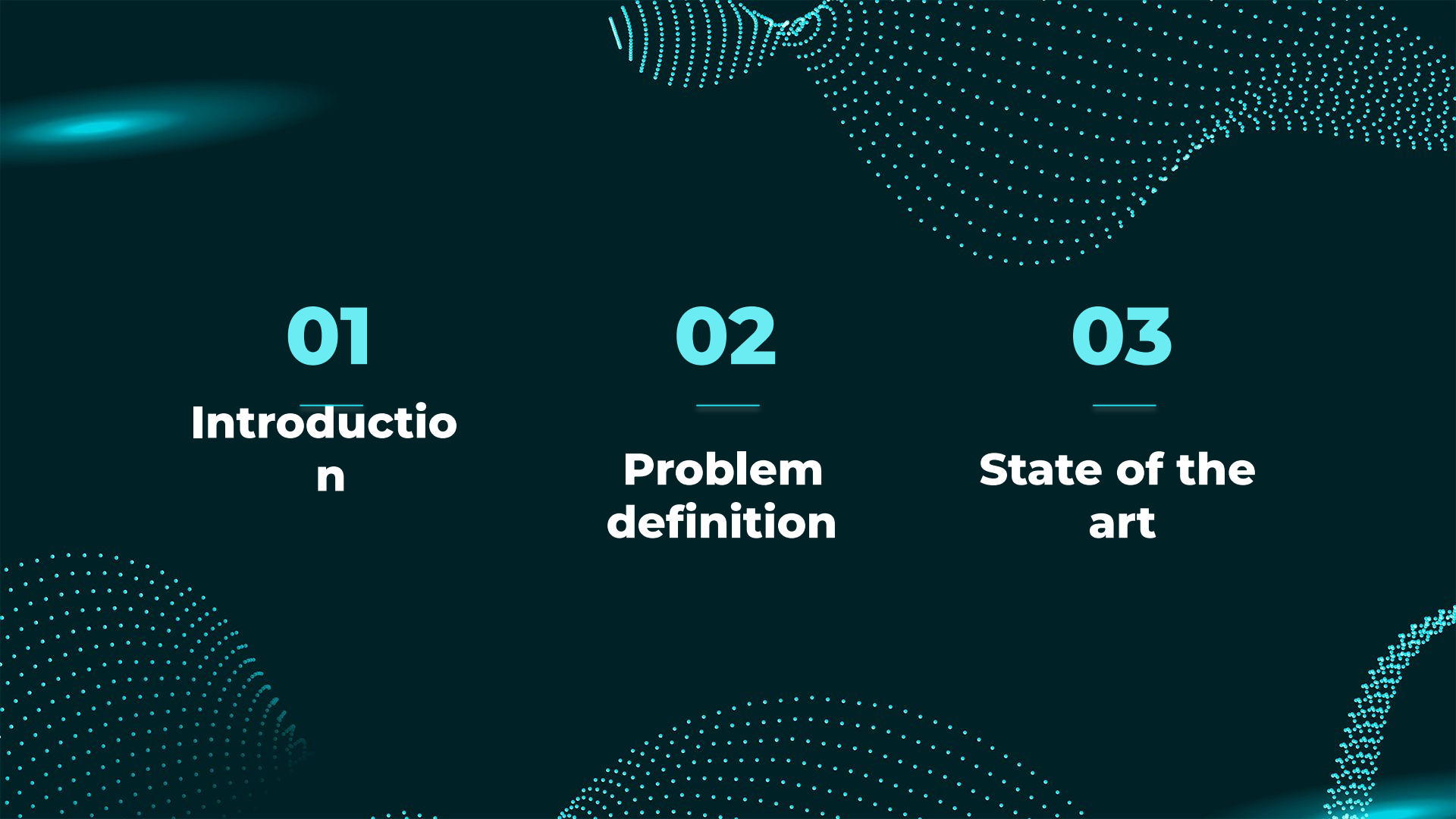


PPP223

A Python SLM Agent Assistant

Team Mobtrap
Balog David
Nimigean Valentin
Șoptelea Sebastian
Szárícs Iulia



01

Introduction

02

**Problem
definition**

03

**State of the
art**



04

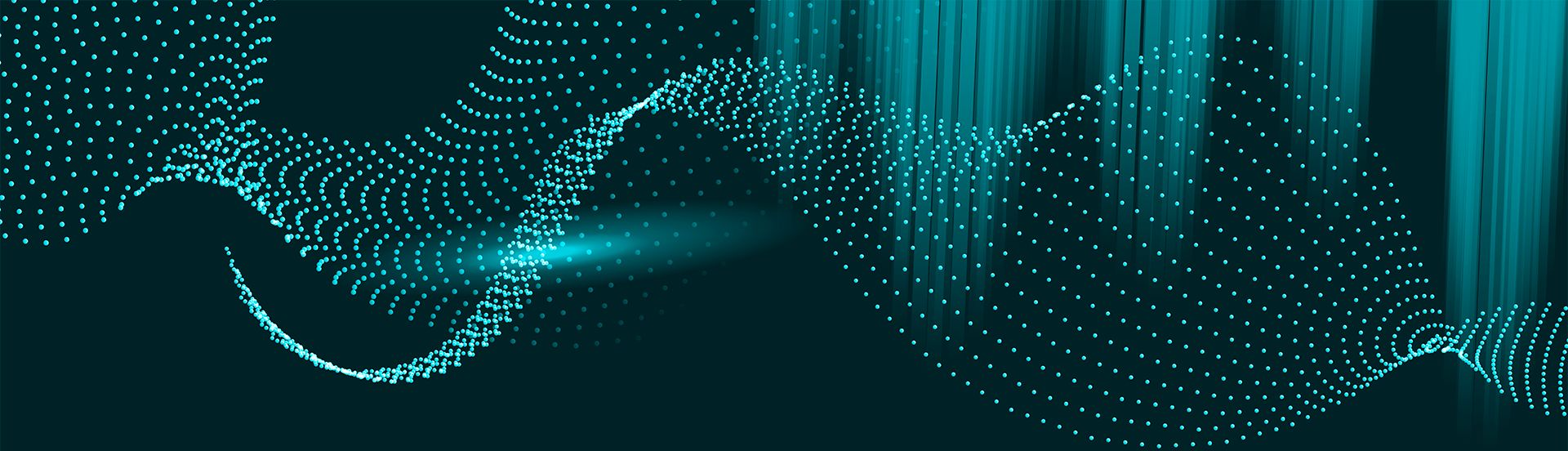
**Our
solution**

05

Experiments

06

Results



01

Introduction

This presentation's role is to show you our work from this semester for the Large Language Models subject. Our project consists of a Language Model Agent capable of understanding Python Code. Our goal was to build an intelligent coding assistant that deeply understands Python repositories, rather than just acting as a simple text-matching tool.

The background is a dark teal color. In the top left corner, there is a cluster of bright teal dots that form a wavy, horizontal pattern. Below this, there are several horizontal, wavy lines made of fine, light teal dots. These lines are arranged in a way that they appear to be flowing or vibrating. The overall effect is a modern, digital, and abstract aesthetic.

02 | **Problem definition**

The challenge of code understanding

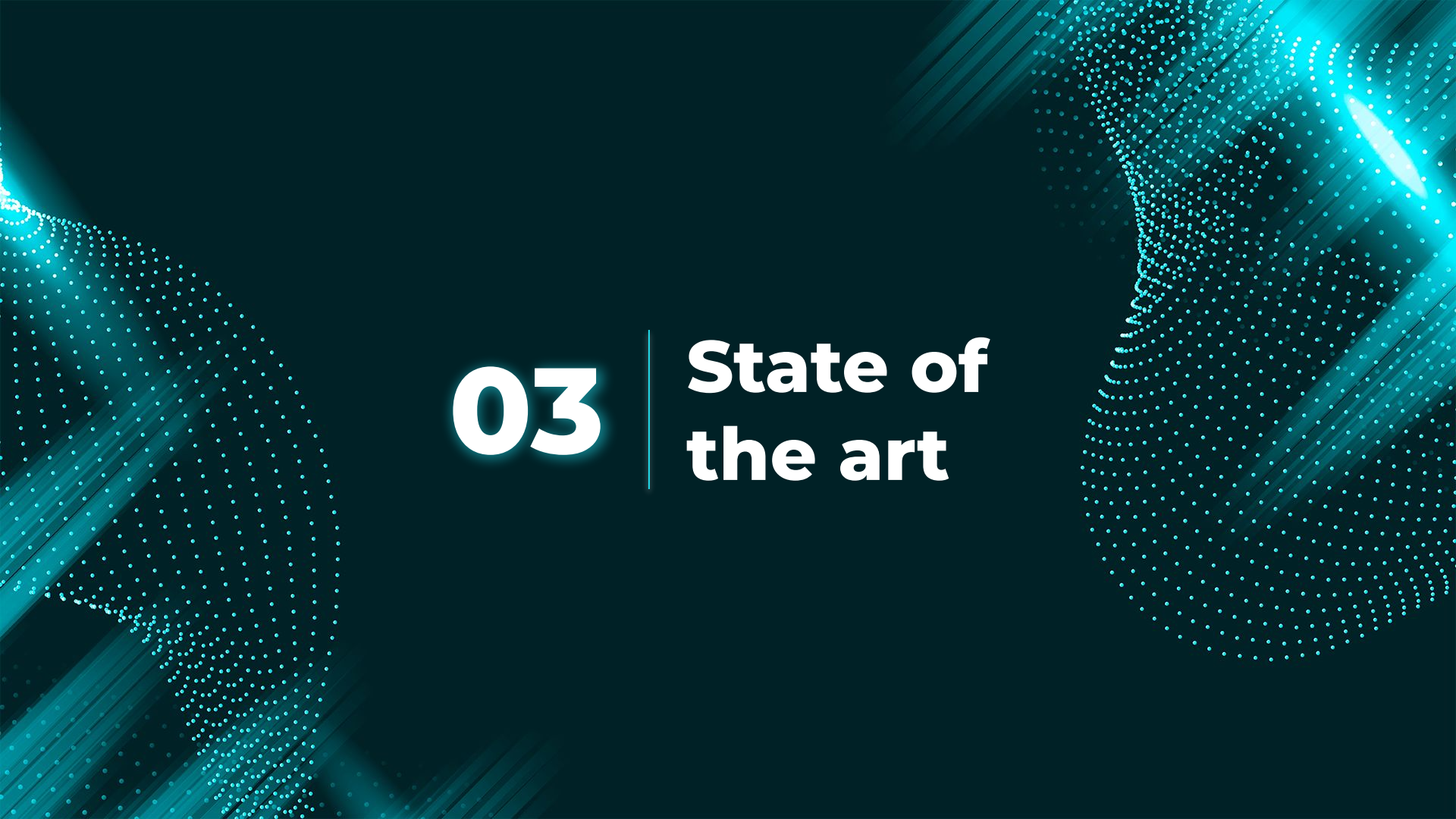
- **Loss of context** : Traditional RAG systems treat code like generic text, losing hierarchical relationships (classes, methods, inheritance)
- **Blind Retrieval**: Basic vector search struggles with code logic - it misses exact variable names or structural dependencies.



The challenge of code understanding

- **Privacy & Resource costs:** Cloud-based LLMs are expensive and pose privacy risks for proprietary codebases.
- **Hallucinations:** Small models often guess code structures when they lack complete repository awareness.





03 | **State of the art**

Current Approaches and Limitations

Approach	Limitation
Standard RAG (Character Chunking)	Breaks logical blocks (e.g., splitting a function in half).
Pure Semantic Search	Great for concepts, terrible for exact keyword/syntax matching.
Large context window	Feeds the whole repo to the model. Very high compute and latency.
Generic instruct models	Struggle to use specialized tool-calling for active repository exploration.

The background features a dark teal gradient. On the left side, there is a large, abstract shape composed of many small, light teal dots, resembling a stylized 'Y' or a branching structure. This shape is overlaid with horizontal teal lines of varying thickness and opacity, creating a layered, digital effect. A bright teal light flare is visible at the bottom of the dot structure.

04

**Our
Solution**

Related Work and Technologies

Component

Approach

Retrieval

Dense vector embeddings (Qdrant) + BM25 keyword fallback (hybrid search)

Parsing

AST-based chunking (tree-sitter for robustness) + Repo Map for navigation

Model

Qwen2.5-Coder (3B + 1.5B variants) via local Ollama

Alignment

QLoRA fine-tuning + DPO for preference optimization

Tooling

Agent framework with ``grep_search`` and ``semantic_search`` tools

Intelligent Retrieval & Specialized Models



Hybrid RAG via Qdrant

Combines Vector Embeddings (semantic meaning) with BM25 Keyword Search (exact syntax/variable matching).



Repo Map Retrieval

Gives the agent a "bird's eye view" of the file structure before fetching specific code.



Active Tool-Calling

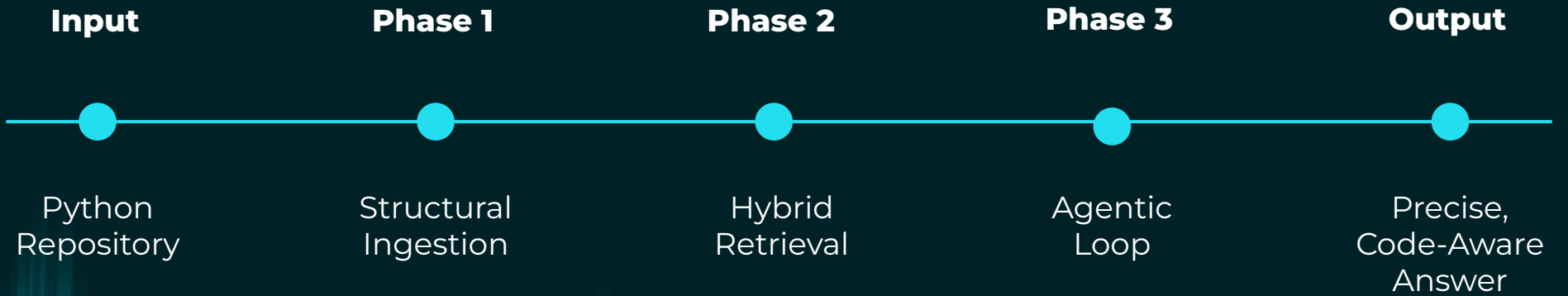
The agent can actively explore the repo



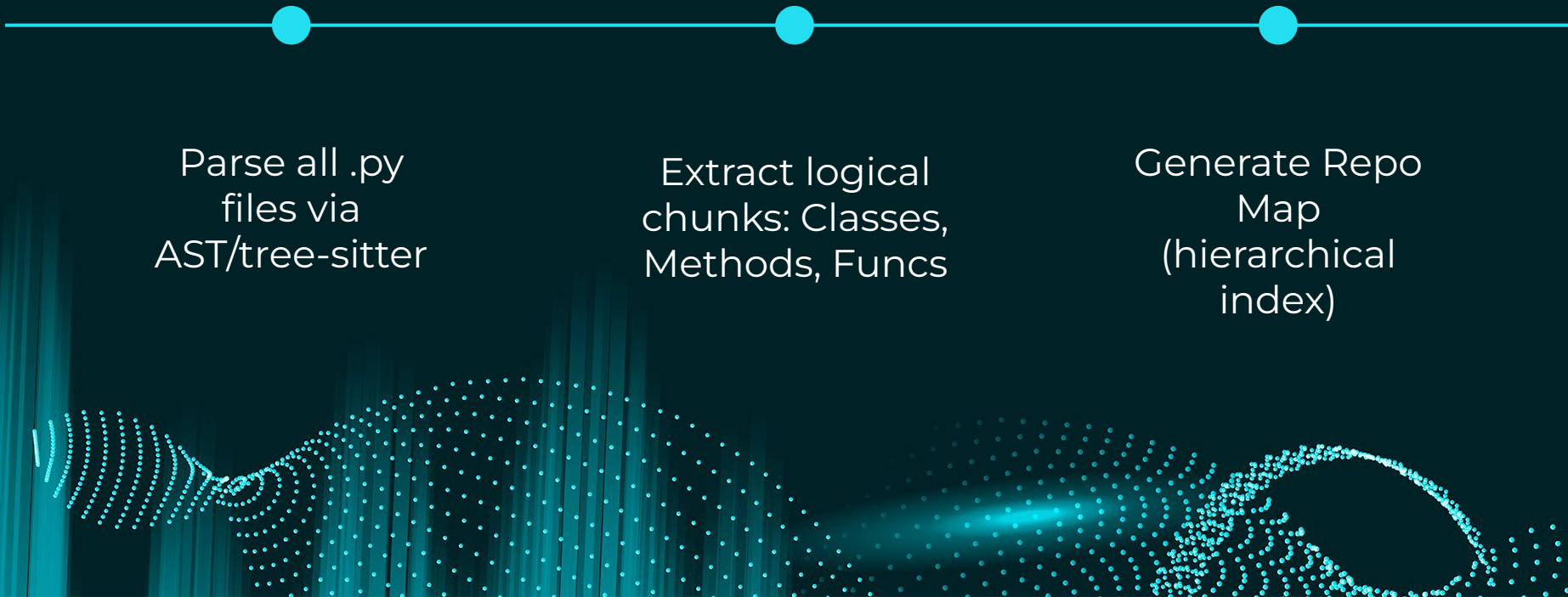
Efficient Fine-Tuning

Using Unsloth QLoRA on Qwen2.5-Coder (1.5B/3B parameters) for local inference, refined with DPO to reduce hallucinations.

Architecture: Three-Pillar System



Phase 1 : Structural Ingestion (AST Chunking)

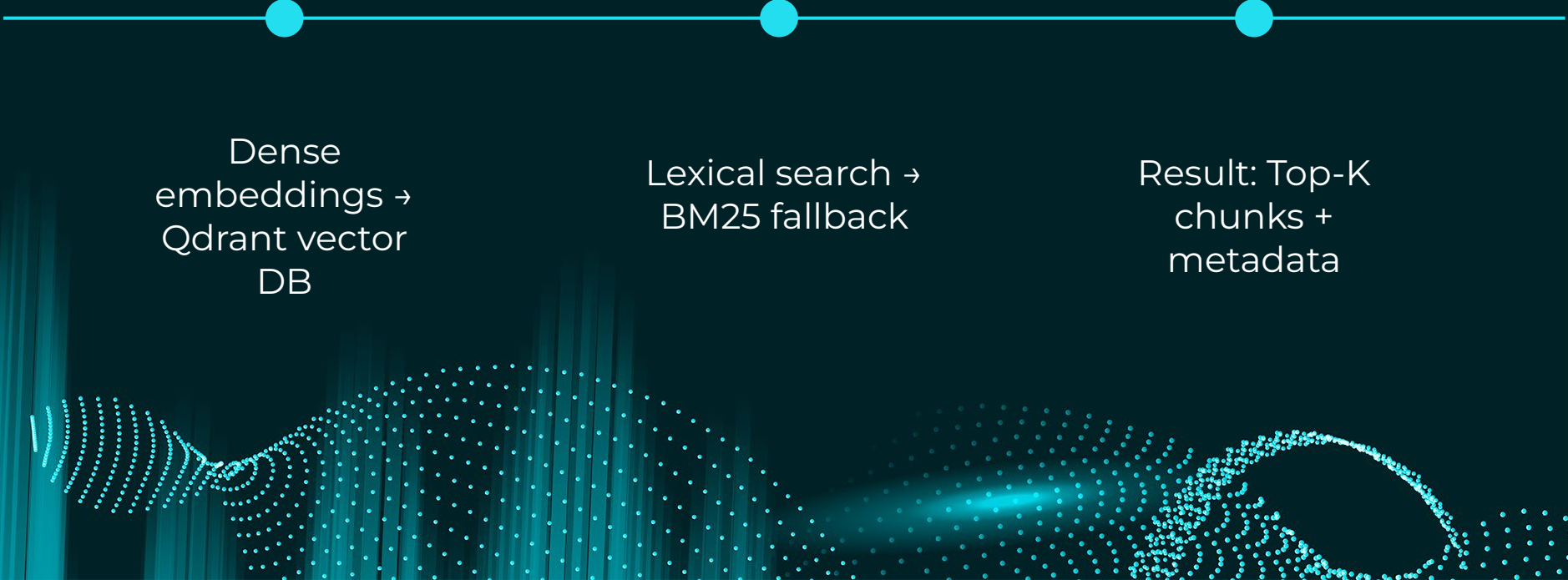


Parse all .py
files via
AST/tree-sitter

Extract logical
chunks: Classes,
Methods, Funcs

Generate Repo
Map
(hierarchical
index)

Phase 2 : Hybrid Retrieval (Vector + BM25)

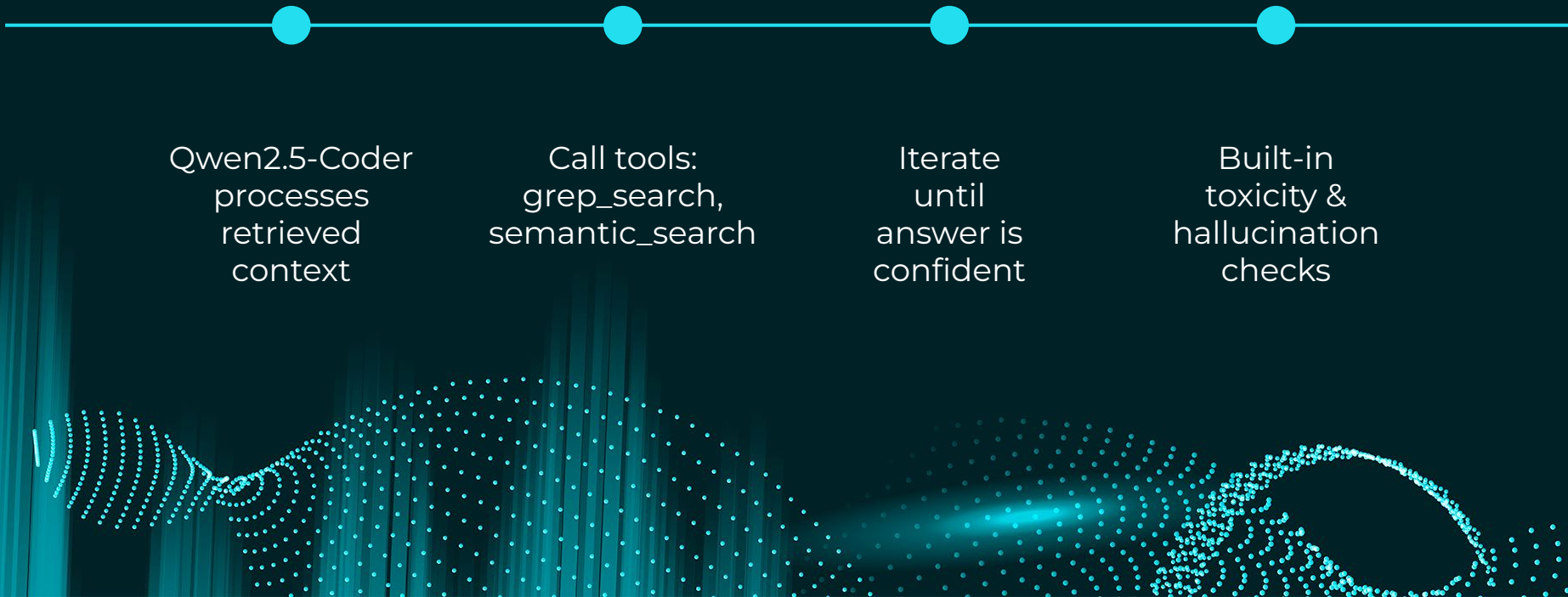


Dense
embeddings →
Qdrant vector
DB

Lexical search →
BM25 fallback

Result: Top-K
chunks +
metadata

Phase 3 : Agentic Loop (SLM Reasoning)



Qwen2.5-Coder
processes
retrieved
context

Call tools:
grep_search,
semantic_search

Iterate
until
answer is
confident

Built-in
toxicity &
hallucination
checks

05

Experiment s



EXPERIMENTS

- We tested the codebase QA agent on internal repository questions that check whether the model can correctly identify relevant files, classes, methods, and implementation details.
- The benchmark covered the main system components: hybrid retrieval, code chunk ingestion, loader behavior, dependency graph logic, exact-match search tools, hallucination handling, and refusal behavior for toxic prompts.

- Metrics Used

Entity Score: Did it name the right class, method, or concept?

File Score: Did it point to the right repository file?

Combined Score: Average correctness across entity+file matching

Penalized Score: Final score after hallucination penalties



06 | Results

Main Result

The full system performed much better than the baseline.

Baseline combined score: 0.42

Full system combined score: 0.92

Full system file score: 1.00

Full system penalized score: 0.92

Good Answers Looked Grounded

The model stopped giving vague answers and started pointing to exact repository evidence.

Example: “The embedding model used for dense vector search is BAAI/bge-small-en-v1.5, configured in rag/retriever.py#L46-L84.”

It Still Had Small Misses

The full system was not perfect on every question.

Two weaker spots:

Tool-call turn limit question scored 0.50

Skipped-directories question scored 0.62

But most questions scored 1.00, and the final overall score stayed strong at 0.92 / 1.00.

Takeaway

The results are good.

The agent became much better at:
finding the correct file,
naming the right class or method,
avoiding unsupported answers,
and giving responses that look like real codebase evidence instead of guesses

DEMO

Python SLM Agent Assistant

powered by Qwen2.5-Coder-3B (Local via Ollama) & Hybrid Qdrant Search



How does AST chunking work?



AST chunking works by dividing a program into smaller, self-contained parts that can be processed independently. It analyzes the syntax tree (AST) of the code and creates chunks based on logical units like functions, classes, method calls, etc., which helps in modularizing the input data for efficient processing.

To understand AST chunking better, you can refer to the `ASTChunker` class definition in the `ingest\chunker.py` file. It iterates through the source code and identifies different levels of logical structures as described below:

1. **Function Definition** (`__init__`): This method sets up the function.
2. **Checking if a chunk is too large** (`_check_large_chunk`): It ensures chunks don't exceed a certain size to reduce memory usage and processing complexity.
3. **Extracting Nested Functions** (`_extract_nested_functions`): By traversing the tree, it looks for complex functions that have sub-functions or nested structures.
4. **Handling Classes** (`_extract_from_class`): This method processes classes, extracting methods and class variables into separate chunks.
5. **Chunking File Content**: The `_chunk_file` method combines the results from above steps to generate final chunks of code that encompass logical units.

Example usage can be found in the `test_chunker.py` file (`<path>/model/tests/test_chunker.py`) where different types of functions, methods, and other constructs are tested within a chunked manner. Essentially, AST chunking is used to optimize data handling by breaking down complex programs into manageable pieces for analysis.

Ask a question about the codebase... (e.g. 'How does AST chunking work?')



Thank you!

We're happy to take any questions

